The Godfather of Talent | Futurense



Kafka Cheat Sheet

Topics, Partitions & Offsets

- **CREATING TOPICS:** using partitions, replication factor

```
kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1
--topic your_topic_name
```

- **LISTING TOPICS**

```
kafka-topics.sh --list --zookeeper localhost:2181
```

- **DESCRIBING TOPICS**

```
kafka-topics.sh --describe --zookeeper localhost:2181 --topic your_topic_name
```

- **DELETING TOPICS**

```
kafka-topics.sh --zookeeper localhost:2181 --delete --topic your_topic_name
```

- **ALTERING TOPICS**

```
kafka-topics.sh --zookeeper localhost:2181 --alter --topic your_topic_name --partitions new_count
```

- **CREATE TOPIC WITH MULTIPLE PARTITIONS**

```
kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 3
--topic your_topic_name
```

- **CHANGING THE RETENTION PERIOD FOR A TOPIC**

```
kafka-topics.sh --bootstrap-server URL --alter --topic topic-name config retention.ms=360000
```

- **PURGE A KAFKA TOPIC:** We can purge a topic by changing the retention time of a topic to one minute

```
kafka-configs.sh --zookeeper localhost:2181 --alter --entity-type topics --entity-name
my-first-topic --add-config retention.ms=1000
```

Wait one minute. Kafka will now remove records older than one minute from the topic. Now delete the retention.ms configuration from the topic so it will default back to the retention configuration of the Kafka cluster

```
kafka-configs.sh --zookeeper localhost:2181 --alter --entity-type topics --entity-name
my-first-topic --delete-config retention.ms
```

- **LIST KAFKA TOPICS THAT HAVE SPECIFIC CONFIGURATION OVERRIDES**

```
kafka-topics --zookeeper zookeeper:2181 --describe --topics-with-overrides
```

- **ALTER A TOPIC'S CONFIGURATION**

```
kafka-configs.sh --alter --zookeeper localhost:2181 --entity-type topics --entity-name
your_topic_name --add-config min.insync.replicas=2
```

- **GET THE EARLIEST OFFSET OF A TOPIC**

```
kafka-run-class.sh kafka.tools.GetOffsetShell --broker-list localhost:9092 --topic topic_name
--time -2
```

- **GET THE LATEST OFFSET IN A TOPIC**

```
kafka-run-class.sh kafka.tools.GetOffsetShell --broker-list localhost:9092 --topic topic_name
--time -1
```





- **RESET THE OFFSET OF A CONSUMER TO THE EARLIEST OFFSET**

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group your_group_id
--reset-offsets --to-earliest --execute --all-topics
```

- **RESET THE OFFSET OF A CONSUMER TO THE LATEST OFFSET**

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group your_group_id
--reset-offsets --to-latest --execute --all-topics
```

- **RESET THE OFFSET OF A CONSUMER TO A SPECIFIC OFFSET**

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group your_group_id
--reset-offsets --to-offset your_desired_offset --execute --topic your_topic_name
```

ZooKeeper Commands used in Kafka Administration

- **START ZOOKEEPER SERVER**

```
zookeeper-server-start.sh config/zookeeper.properties
```

- **STOP ZOOKEEPER SERVER**

```
zookeeper-server-stop.sh
```

- **CONNECT TO ZOOKEEPER SHELL**

```
zkCli.sh
```

- **CREATE A ZNODE (ZOOKEEPER NODE)**

```
create /path value
```

- **WATCH A ZNODE FOR CHANGES**

```
get /path true
```

- **SET/UPDATE THE DATA OF A ZNODE**

```
set /path value
```

- **LIST THE CHILD ZNODES UNDER A PATH**

```
ls /path
```

- **DELETE A ZNODE**

```
delete /path
```

- **GET THE DATA OF A ZNODE**

```
get /path
```

- **MONITOR THE STATUS OF CONNECTED CLIENTS**

```
mntr
```

- **SET ACL (ACCESS CONTROL LIST) FOR A ZNODE**

```
setAcl /path acl
```

- **GET THE LIST OF ACLS FOR A ZNODE**

```
getAcl /path
```

- **SET A WATCH ON A ZNODE FOR A SPECIFIC EVENT TYPE**

```
wchc /path
```

- **CHECK THE LEADER FOR A PARTICULAR PARTITION**

```
get /brokers/topics/<topic_name>/partitions/<partition_id>/state
```

- **GET THE LIST OF ACTIVE BROKERS IN THE KAFKA CLUSTER**

```
ls /brokers/ids
```

- **GET DETAILED INFORMATION ABOUT A BROKER**

```
get /brokers/ids/<broker_id>
```





Kafka Producer API

- **SENDING MESSAGES WITH KAFKA CONSOLE PRODUCER**

```
echo "Hello, World" | kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic
```

- **SEND MESSAGES FROM A FILE**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic < messages.txt
```

- **SEND MESSAGES WITH KEYS**

```
echo "key1:value1" | kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic
--property "parse.key=true" --property "key.separator=:"
```

- **PRODUCER WITH ACKNOWLEDGEMENTS**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --request-required-acks 1
```

- **PRODUCER WITH COMPRESSION**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --compression-codec gzip
```

- **PRODUCER WITH BATCH SIZE**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --batch-size 100
```

- **SPECIFY KAFKA SERVER PROPERTIES**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --producer-property
<property_name>=<property_value>
```

- **PRODUCE MESSAGES WITH A CUSTOM PARTITIONER**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --property
"partitioner.class=<custom_partitioner_class>"
```

- **PRODUCE MESSAGES WITH A SPECIFIC PARTITION**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --property
"parse.key=true" --property "key.separator=:" --property "partition=<partition_number>"
```

- **PRODUCE MESSAGES WITH A SPECIFIC MESSAGE KEY**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --property
"parse.key=true" --property "key.separator=:" --property "key=<message_key>"
```

- **PRODUCE MESSAGES WITH A SPECIFIC MESSAGE TIMESTAMP**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --property
"parse.key=true" --property "key.separator=:" --property "timestamp=<timestamp>"
```

- **PRODUCE MESSAGES WITH CUSTOM MESSAGE HEADERS**

```
kafka-console-producer.sh --broker-list localhost:9092 --topic my_topic --property
"parse.key=true" --property "key.separator=:" --property "header.<header_key>=<header_value>"
```

Kafka Consumer API

- **STARTING A CONSUMER**

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic my_topic --from-beginning
```





- **STARTING A CONSUMER WITH SPECIFIC CONFIGURATIONS**

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic my_topic --consumer-property group.id=my_group --consumer-property client.id=my_client
```

- **STARTING A CONSUMER WITH A CUSTOM DESERIALIZER**

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic my_topic --formatter kafka.tools.DefaultMessageFormatter --property key.deserializer=org.apache.kafka.common.serialization.StringDeserializer --property value.deserializer=custom.deserializer.Class
```

- **READING DATA FROM KAFKA TOPIC**

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic my_topic --from-beginning
```

- **USING A CONSUMER GROUP**

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic my_topic --group my_group
```

- **READING DATA FROM KAFKA TOPIC WITH OFFSET**

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic my_topic --offset 5 --partition 0
```

- **LISTING CONSUMER GROUPS**

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list
```

- **DESCRIBING CONSUMER GROUP**

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group my_group
```

- **READING FROM A SPECIFIC OFFSET**

```
kafka-console-consumer --bootstrap-server localhost:9092 --topic my_topic --offset 5 --partition 0
```

Broker

- **START A KAFKA BROKER**

```
./bin/kafka-server-start.sh config/server.properties
```

- **LIST ALL KAFKA BROKERS**

```
./bin/zookeeper-shell.sh localhost:2181 ls /brokers/ids
```

- **STOP A KAFKA BROKER**

```
./bin/kafka-server-stop.sh config/server.properties
```

- **CHECKING THE HEALTH OF THE BROKER**

```
./kafka-topics.sh --zookeeper localhost:2181 --describe --under-replicated-partitions
```

- **ADDING A BROKER**

```
./kafka-server-start.sh config/server-1.properties
```





Implementing Kafka Single Node

1. START ZOOKEEPER SERVER

```
cd /usr/local/kafka
bin/zookeeper-server-start.sh config/zookeeper.properties
```

2. START KAFKA SERVER

```
bin/kafka-server-start.sh config/server.properties
```

3. CREATE TOPIC

```
bin/kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1
--topic test
```

4. PRODUCE MESSAGES

```
bin/kafka-console-producer.sh --broker-list localhost:9092 --topic test
```

5. CONSUME MESSAGES

```
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test --from-beginning
```

Implementing Kafka Multi Node

1. START ZOOKEEPER SERVER

```
cd /usr/local/kafka
bin/zookeeper-server-start.sh config/zookeeper.properties
```

2. CONFIGURE KAFKA BROKERS

For each broker, you will need to create a copy of the **server.properties** file and modify the following properties:

- `broker.id` : Unique integer for each broker
- `listeners` : Broker will bind to this. Use the IP address or hostname of the machine
- `log.dirs` : Directory for the log data
- `zookeeper.connect` : Comma-separated string containing the IP addresses of all Zookeeper instances

Example:

```
broker.id=1
listeners=PLAINTEXT://:9092
log.dirs=/usr/local/kafka/logs-1
zookeeper.connect=zookeeper1:2181,zookeeper2:2181,zookeeper3:2181
```

3. START KAFKA SERVER

```
bin/kafka-server-start.sh config/server-1.properties
```

4. CREATE TOPIC WITH REPLICATION

```
bin/kafka-topics.sh --create --zookeeper zookeeper1:2181,zookeeper2:2181,zookeeper3:2181
--replication-factor 3 --partitions 1 --topic test
```

5. PRODUCE MESSAGES

```
bin/kafka-console-producer.sh --broker-list localhost:9092 --topic test
```

6. CONSUME MESSAGES

```
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test --from-beginning
```





Kafka Streams (Python)

1. SETTING UP KAFKA STREAMS

```
import faust
app = faust.App('my-first-streams-application', broker='kafka://kafka-broker1:9092')
```

2. CREATING A STREAM

```
txtLinesTopic=app.topic('TextLinesTopic')
@app.agent(txtLinesTopic)
async def process(stream):
    async for value in stream:
        print(value)
```

3. STARTING A STREAM (In shell)

```
faust -A your_python_file worker
```

4. APPLYING TRANSFORMATIONS TO A STREAM

```
@app.agent(txtLinesTopic)
async def process(stream):
    async for value in stream:
        print(value.upper())
```

4. SENDING DATA TO A STREAM

```
await txtLinesTopic.send(value='your message')
```

Avro and Schema Registry

AVRO

```
#Define an Avro schema
{
  "type": "record",
  "name": "User",
  "fields": [
    {"name": "name", "type": "string"},
    {"name": "favorite_number",  "type": ["int", "null"]},
    {"name": "favorite_color", "type": ["string", "null"]}
  ]
}
```

- **KAFKA-AVRO-CONSOLE-PRODUCER AND KAFKA-AVRO-CONSOLE-CONSUMER**
  - To produce and consume messages with Avro schemas, you can use the kafka-avro-console-producer and kafka-avro-console-consumer commands.

```
# Produce a message with an Avro schema
echo '{"name": "John Doe", "favorite_number": 7, "favorite_color": "red"}' |
kafka-avro-console-producer \
  --broker-list localhost:9092 --topic test_topic \
  --property schema.registry.url=http://localhost:8081 \
  --property value.schema='{"type":"record","name":"User","fields":[{"name":"name",
"type":"string"},{"name":"favorite_number","type":["int","null"]},{"name":"favorite_color",
"type":["string","null"]}]}'

# Consume messages with an Avro schema
kafka-avro-console-consumer --topic test_topic \
  --from-beginning \
  --bootstrap-server localhost:9092 \
  --property schema.registry.url=http://localhost:8081
```





Use of ChatGPT in Kafka

Integrating ChatGPT in Kafka

- **PYTHON PROGRAM THAT USES THE KAFKA PRODUCER AND CONSUMER APIS TO SEND AND RECEIVE MESSAGES TO / FROM A KAFKA TOPIC, AND WITHIN THE SAME PROGRAM**
  - To use GPT-3, you have to make HTTP requests to OpenAI's servers, so it can't be done with bash scripting alone. We have to use a programming language that supports HTTP and JSON, like Python

```
from kafka import KafkaConsumer, KafkaProducer
from openai import GPT3, api_key
#Configure Kafka consumer and producer
producer = KafkaProducer(bootstrap_servers='localhost:9092')
consumer = KafkaConsumer('my-topic', bootstrap_servers='localhost:9092')
#Set your OpenAI API key
api_key('your-openai-api-key')
#Consume messages from Kafka
for message in consumer:
    #Send message to GPT-3
    response = GPT3.complete(prompt=message.value, max_tokens=60)
    #Send response back to Kafka
    producer.send('response-topic', response.choices[0].text.strip())
```

ChatGPT and CodeGPT Prompts for Kafka

- **SETTING UP KAFKA PRODUCERS AND CONSUMERS IN PYTHON**
  - **ChatGPT:** *"Provide me a detailed explanation and an example code of how to set up Kafka Producers and Consumers using Python?"*
  - **CodeGPT:** *"Generate Python code to set up a Kafka Producer that sends a message to a specified topic, and a Kafka Consumer that reads this message."*
- **IMPLEMENTING KAFKA STREAMS IN PYTHON**
  - **ChatGPT:** *"Can you explain the concept of Kafka Streams and provide an example of how to implement it in Python?"*
  - **CodeGPT:** *"Generate Python code that implements a simple Kafka Stream."*
- **KAFKA PARTITIONING IN PYTHON**
  - **ChatGPT:** *"Could you explain the concept of Kafka partitioning and provide an example of how to create partitions in a Kafka topic using Python?"*
  - **CodeGPT:** *"Generate Python code to create a Kafka topic with multiple partitions."*
- **WORKING WITH KAFKA TOPICS**
  - **ChatGPT:** *"How do I create, list, and delete Kafka topics using Python? Could you provide examples"*
  - **CodeGPT:** *"I need Python code to create, list, and delete Kafka topics."*





- **KAFKA MESSAGE COMPRESSION**
  - **ChatGPT:** *"How do I implement message compression in Kafka using Python? Can you provide an explanation and example?"*
  - **CodeGPT:** *"Generate Python code to produce and consume messages in Kafka with compression enabled."*
- **KAFKA SECURITY (SSL AND SASL)**
  - **ChatGPT:** *"Could you explain how to secure Kafka clusters using SSL and SASL in Python? Could you provide some examples?"*
  - **CodeGPT:** *"Generate Python code to produce and consume messages from a Kafka cluster secured with SSL/SASL."*
- **IMPLEMENTING KAFKA CONNECT**
  - **ChatGPT:** *"Explain the role of Kafka Connect in data pipelines and its implementation process."*
  - **CodeGPT:** *"Generate a configuration for Kafka Connect Source and Sink connectors to stream data from a database to a Kafka topic and vice versa."*
- **KAFKA MONITORING AND ADMINISTRATION**
  - **ChatGPT:** *"Could you discuss various options available for monitoring and administrating a Kafka cluster? Include key performance metrics and administrative commands."*
  - **CodeGPT:** *"Please provide examples of using Kafka administrative client in Python to retrieve cluster information, topic details, consumer group details, etc."*
- **KAFKA TRANSACTIONS**
  - **ChatGPT:** *"Please provide a detailed explanation of transactions in Kafka. How do they ensure exactly-once processing semantics?"*
  - **CodeGPT:** *"Provide an example of a Kafka Producer and Consumer in Python that use transactions to ensure exactly-once semantics."*
- **KAFKA AND KUBERNETES**
  - **ChatGPT:** *"Could you explain how to deploy and manage Kafka on Kubernetes? Include concepts like StatefulSets, Persistent Volumes, and Helm charts."*
  - **CodeGPT:** *"Create a sample Helm values.yaml configuration file for deploying a Kafka cluster on Kubernetes."*

